Telecom Messaging DevOps Platform: Low Level Design (LLD)

Course Name: DevOps Project **Institution Name:** Medicaps University - Datagami Skill Based Course

---

Sr no	Student Name	Enrolment Number
1	Sanand Alshi	EN22CS301859
2	Sanidhya Yadav	EN22CS301866
3	Sanidhya Somani	EN22CS301865
4	Samiksha Saxena	EN22CS301853
5	Sarthak Chhajed	EN22CS301874

---

1. Introduction

1.1. Scope of the document

This document serves as a detailed Low Level Design (LLD) for the Telecom Messaging DevOps Platform. It outlines the technical architecture, component design, and automated workflows required to build and deploy a simulated telecom backend system. It covers everything from the initial code structure to the final Kubernetes orchestration.

1.2. Intended Audience

- **DevOps Engineers**: Responsible for implementing the CI/CD pipeline and IaC scripts.
- **Developers**: Focused on the Flask-based API logic and containerization.
- **System Administrators**: Tasked with monitoring the Kubernetes cluster and service health.

1.3. System Overview

The project addresses the challenge of manual, error-prone configurations in telecom environments. By utilizing Infrastructure as Code (IaC) and containerization, the system automates the setup of a telecom messaging API that handles SMS, user management, and delivery reporting. This ensures repeatable deployments and consistent environments across all stages of production.

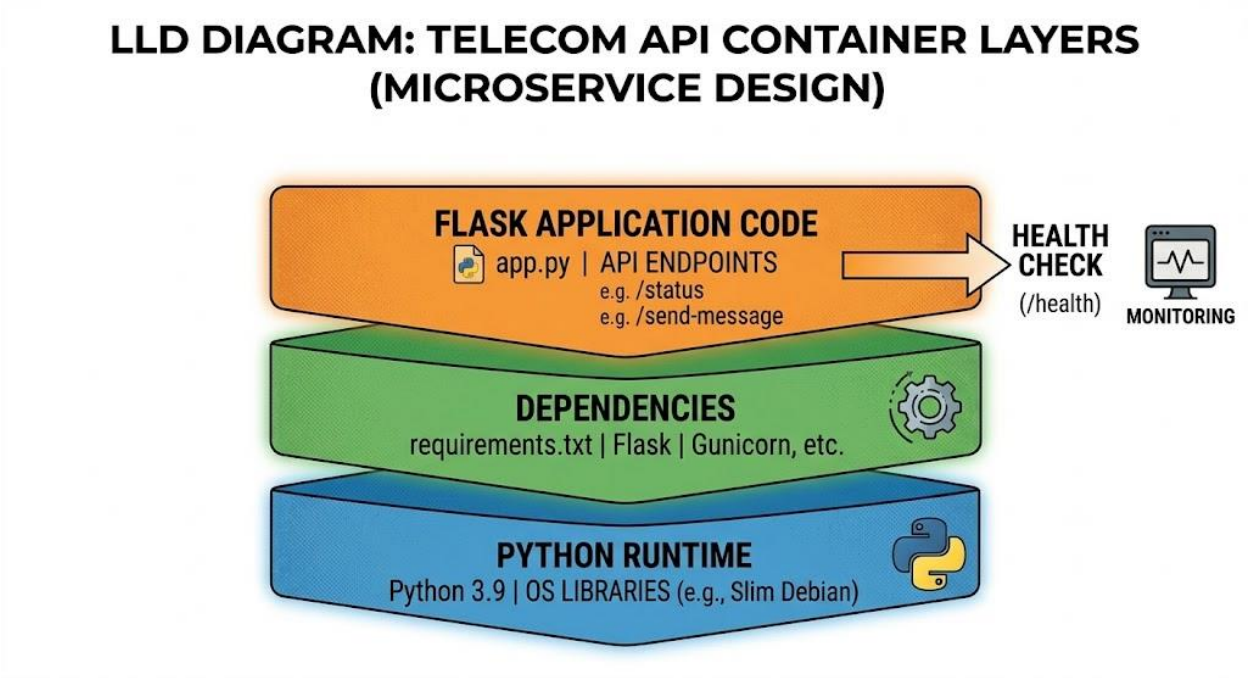
---

2. System Design

2.1. Application Design

The core of the system is a **Telecom Messaging API** built using the **Flask framework**.

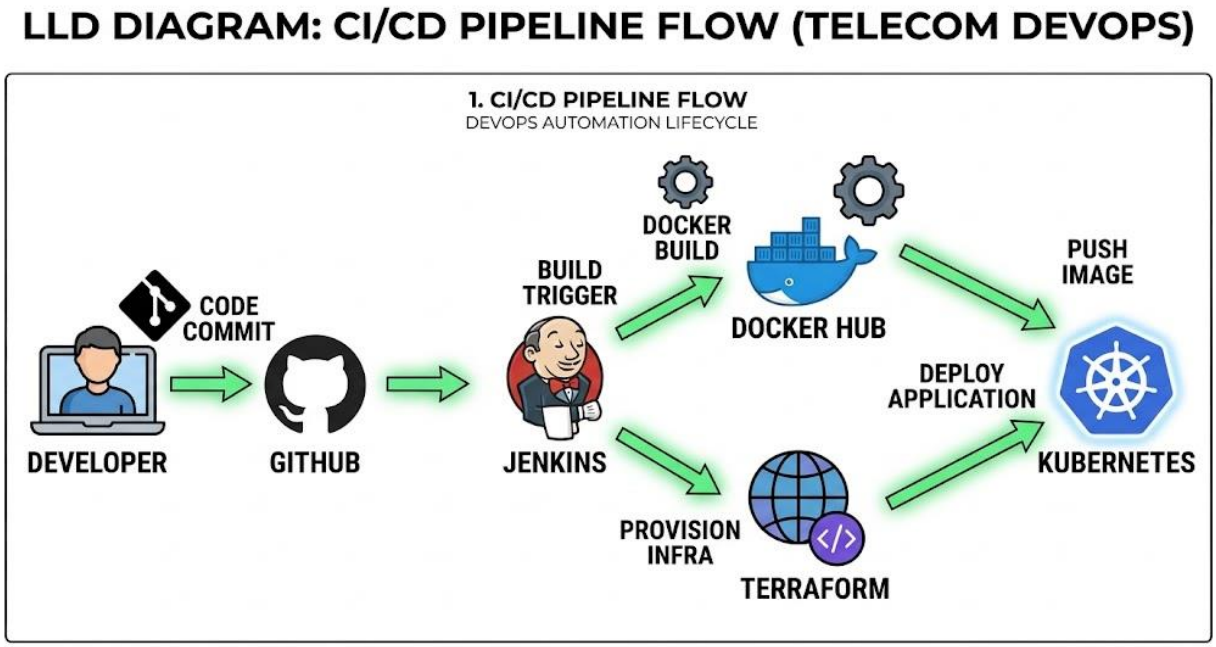
- **Functionality**: The API simulates backend operations like sending SMS, managing users, and checking delivery reports.
- **Environment**: It runs within a Python runtime environment, packaged with all necessary dependencies in a Docker container.
- **Port Configuration**: The application is configured to listen on **port 5000** for all incoming REST API requests.



2.2. Process Flow

The system follows a strict automated pipeline to move code from development to a live environment.

1. **Code Commit**: A developer pushes new features or fixes to the **GitHub** repository.
2. **Jenkins Trigger**: The **Jenkins CI/CD pipeline** starts automatically upon detecting code changes.
3. **Build Phase**: Jenkins executes the `Dockerfile` to build a new container image.
4. **Registry Push**: The verified Docker image is pushed to **Docker Hub** for versioned storage.
5. **Infrastructure Provisioning**: **Terraform** initializes and applies configurations to setup the Kubernetes environment.
6. **Deployment**: **Kubernetes** pulls the latest image and deploys it as a scalable service.



2.3. Information Flow

The flow of information ensures that end-users can interact with the telecom services reliably.

- **Request Entry**: Users send requests to the Kubernetes cluster via a **NodePort** service.
- **Access Point**: The service is accessible at `http://localhost:30007`.
- **Internal Routing**: Kubernetes routes traffic to the active application pods.
- **Feedback Loop**: The application provides a `/health` endpoint that Kubernetes queries to ensure the container is functioning correctly.

---

3. Component Design

3.1. Project Structure

The project is organized into modular directories to maintain a clean separation of concerns.

Path/File	Description
`app/app.py`	The main Flask application containing API logic.
`app/requirements.txt`	List of Python packages (e.g., Flask) required for the app.
`kubernetes/deployment.yaml`	Defines pod replicas and container specifications.
`kubernetes/service.yaml`	Configures the NodePort exposure for external traffic.
`terraform/main.tf`	IaC configuration for provisioning Kubernetes resources.
`Dockerfile`	Set of instructions for building the Docker image.
`Jenkinsfile`	The pipeline-as-code script defining CI/CD stages.

3.2. Detailed Pipeline Stages

The Jenkins pipeline is the "Delivery Manager" of the project.

- **Stage 1: Checkout**: Pulls the latest source code from the Git repository.
- **Stage 2: Build**: Creates a container image using the local Docker daemon.
- **Stage 3: Tagging**: Assigns a version or 'latest' tag to the Docker image.
- **Stage 4: Push**: Uploads the image to the central Docker Hub registry.
- **Stage 5: IaC Apply**: Executes Terraform to ensure the infrastructure matches the code.
- **Stage 6: K8s Deploy**: Updates the Kubernetes cluster to use the new image.

---

4. Technical Specifications

4.1. API Catalogue

The following REST endpoints are implemented to simulate a real-world telecom backend.

Endpoint	Method	Detailed Description
`/status`	GET	Checks if the general service is operational.

Endpoint	Method	Detailed Description
`/health`	GET	Specialized health check for Kubernetes monitoring.
`/send-message`	POST	Submits a simulated SMS message to the system.
`/check-balance`	GET	Returns the current SMS credit balance for a user.
`/create-user`	POST	Logic to add a new telecom subscriber.
`/users`	GET	Displays a list of all registered users.
`/delete-user/{id}`	DELETE	Removes a specific user by their unique ID.
`/recharge`	POST	Adds credits to a user's account for messaging.
`/message-status/{id}`	GET	Tracks if a message is delivered or pending.
`/delivery-report`	GET	Provides statistical data on message success rates.
`/service-info`	GET	Provides system-level metadata and info.

4.2. Infrastructure as Code (IaC) Components

Terraform is used to define and manage the following resources automatically.

- **Kubernetes Deployments**: Manages the lifecycle of the pods, ensuring high availability.
- **Kubernetes Services**: Provides a stable network identity (NodePort) for the pods.
- **Resource Limits**: Defines CPU and memory constraints for the containers.

---

5. Key Design Considerations

5.1. Reliability and Scalability

- **Automation**: By removing manual steps, we eliminate "human-in-the-loop" errors such as missing dependencies.
- **Orchestration**: Kubernetes handles container failures by automatically restarting pods, ensuring the API stays online.
- **Consistency**: Docker ensures that "it works on my machine" translates to "it works in production".

5.2. Analogy of the System

To simplify the complex DevOps workflow, consider this food delivery analogy:

- **Chef (Developer)**: Prepares the "food" (code).
- **Packaging (Docker)**: Wraps the food for safe transport.

- **Warehouse (Docker Hub)**: Stores the packaged goods.
- **Delivery Manager (Jenkins)**: Coordinates the entire movement.
- **Kitchen Builder (Terraform)**: Sets up the physical environment to work in.
- **Delivery Fleet (Kubernetes)**: Ensures the final product reaches the customer (user).

---

6. Conclusion

The Telecom Messaging DevOps Platform demonstrates a modern, high-fidelity approach to software delivery. By integrating Git, Jenkins, Docker, Terraform, and Kubernetes, the project showcases how telecom companies can achieve **Scalability**, **Reliability**, and **Automation**. This implementation follows industry-standard practices used to manage large-scale production environments in the cloud era.

7. References

1. Telecom DevOps Project - Team Understanding Guide.
2. Telecom Messaging DevOps Platform - Low Level Design (LLD).
3. Medicaps University - Project LLD Template.